HW06 — Mandatory AI Critique

AI was useful for producing a broad first pass of equivalence classes, boundary ideas, payload variations, and draft Postman assertions. It was much less reliable for the details that determine whether an API test is meaningful. A model can name a three-attempt lockout boundary, for example, while still failing to preserve account state between requests or to distinguish the specified result from the implementation’s actual behavior. It also tends to assume conventional REST validation outcomes instead of verifying this Express application’s real response schema and persistence behavior.

The most important incompleteness appeared in stateful and security-sensitive paths. A one-request checkout design does not prove that the server calculates from the cart or clears it afterwards. Likewise, a state-transition table alone does not prove role enforcement: the request must be sent with a real customer token against an order in the exact starting state. AI also initially encouraged summary numbers such as “120 tests passed” without a defensible relationship to the raw Newman request and assertion counts. Those are reporting defects, not merely wording issues.

The reason is that an LLM predicts plausible test documentation from text; it does not automatically own a clean database fixture, execute the SUT, or know whether an observed result is a requirement violation. The collaboration principle I learned is therefore to use AI as a structured design accelerator, then make the human tester responsible for the oracle, isolation, execution evidence, severity decision, and final claims. Raw logs and reproducible steps are the authority; polished prose is not.